```
import nltk
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from nltk.corpus import movie_reviews, stopwords
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
nltk.download('movie_reviews')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package movie_reviews to /root/nltk_data...
[nltk_data]   Unzipping corpora/movie_reviews.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

```
# Positive reviews
positive_reviews = [
    movie_reviews.raw(fileid)
    for fileid in movie_reviews.fileids('pos')
]

# Negative reviews
negative_reviews = [
    movie_reviews.raw(fileid)
    for fileid in movie_reviews.fileids('neg')
]

print("Positive reviews:", len(positive_reviews))
print("Negative reviews:", len(negative_reviews))
```

```
Positive reviews: 1000
Negative reviews: 1000
```

```
stop_words = stopwords.words('english')
```

```
tfidf_pos = TfidfVectorizer(stop_words=stop_words, max_features=5000)
tfidf_pos_matrix = tfidf_pos.fit_transform(positive_reviews)
```

```
tfidf_neg = TfidfVectorizer(stop_words=stop_words, max_features=5000)
tfidf_neg_matrix = tfidf_neg.fit_transform(negative_reviews)
```

```
def get_top_terms(tfidf_matrix, vectorizer, top_n=15):
    avg_tfidf = np.mean(tfidf_matrix.toarray(), axis=0)
    terms = vectorizer.get_feature_names_out()
    tfidf_scores = dict(zip(terms, avg_tfidf))
    sorted_terms = sorted(tfidf_scores.items(), key=lambda x: x[1], reverse=True)
    return sorted_terms[:top_n]
```

```
top_pos_terms = get_top_terms(tfidf_pos_matrix, tfidf_pos)
top_neg_terms = get_top_terms(tfidf_neg_matrix, tfidf_neg)
```

```
print("Top 15 Positive TF-IDF Terms:")
for term, score in top_pos_terms:
    print(term, round(score, 4))

print("\nTop 15 Negative TF-IDF Terms:")
for term, score in top_neg_terms:
    print(term, round(score, 4))
```

```
Top 15 Positive TF-IDF Terms:
film 0.0609
movie 0.0387
one 0.0362
like 0.0256
story 0.0209
good 0.0207
life 0.0197
time 0.0197
well 0.0193
also 0.0193
even 0.0184
character 0.0183
characters 0.0181
two 0.0176
```

```
much 0.0175

Top 15 Negative TF-IDF Terms:
film 0.0587
movie 0.0479
one 0.0379
like 0.0281
even 0.0228
good 0.0214
bad 0.0207
time 0.0205
would 0.0198
get 0.0196
story 0.0195
much 0.019
plot 0.0187
characters 0.0181
character 0.0181
```

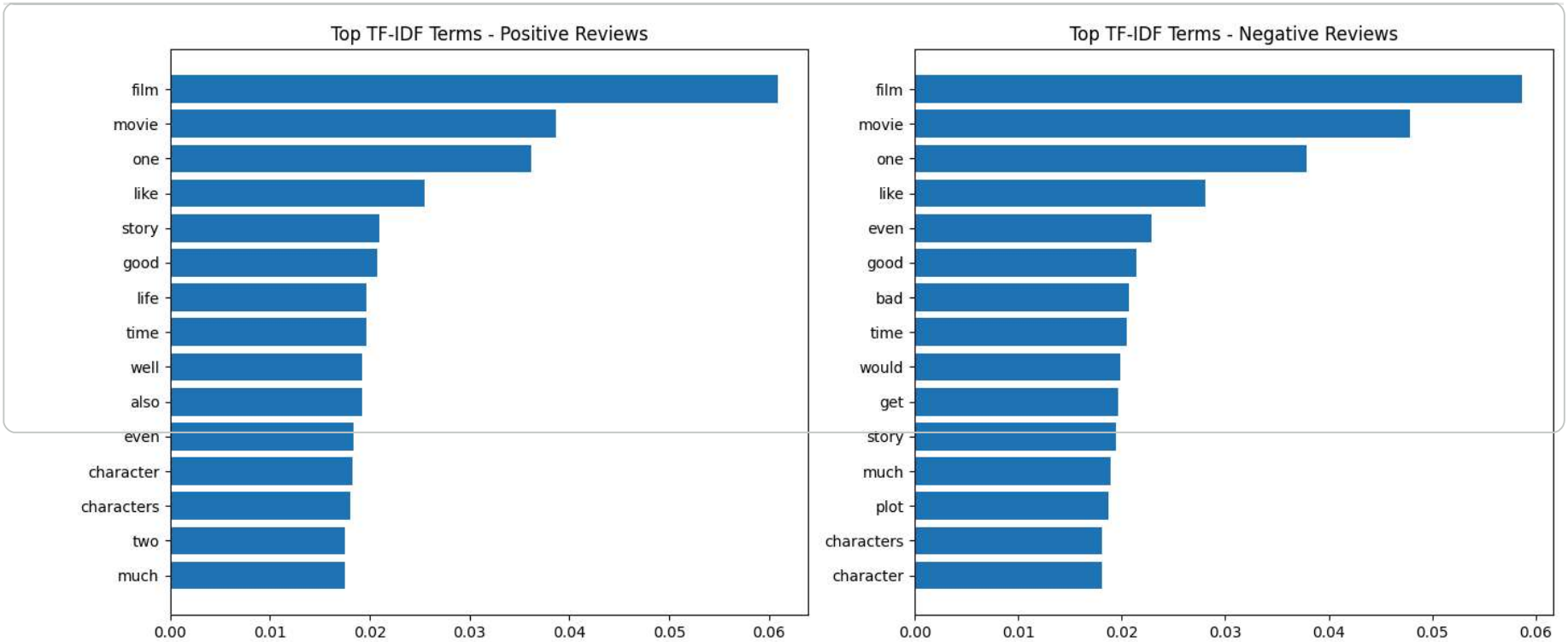
```
pos_words, pos_scores = zip(*top_pos_terms)
neg_words, neg_scores = zip(*top_neg_terms)

plt.figure(figsize=(14,6))

plt.subplot(1,2,1)
plt.barh(pos_words, pos_scores)
plt.title("Top TF-IDF Terms - Positive Reviews")
plt.gca().invert_yaxis()

plt.subplot(1,2,2)
plt.barh(neg_words, neg_scores)
plt.title("Top TF-IDF Terms - Negative Reviews")
plt.gca().invert_yaxis()

plt.tight_layout()
plt.show()
```


Top TF-IDF Terms - Positive Reviews
film
movie
one
like
story
good
life
time
well
also
even
character
characters
two
much
0.00
0.01
0.02
0.03
0.04
0.05
0.06
Top TF-IDF Terms - Negative Reviews
film
movie
one
like
even
good
bad
time
would
get
story
much
plot
characters
character
0.00
0.01
0.02
0.03
0.04
0.05
0.06